

Contents

CUAD Legal Document Analysis — Complete Project Report	1
0. Overview	1
Step 1 — Data source	2
Step 2 — Data preparation (wide → long melt)	2
Step 3 — Exploratory Data Analysis (EDA)	2
Step 4 — The 80/20 train/test split	2
Step 5 — Training Stage 2A: Presence classifier	3
5.1 Strong baseline first — TF-IDF + Logistic Regression (trained on 80%)	3
5.2 The retrieval-ceiling check (why we did not window naively)	3
5.3 The fix — sliding-window max-pool (Multiple-Instance Learning)	3
5.4 The winning model — complementary AND-ensemble	3
5.5 How much of that ranking is statistically real?	4
5.6 Two controls the bootstrap cannot run	4
5.7 Does the AND rule punish rare categories?	5
Step 6 — Training Stage 2B: Span extractor	5
Step 7 — Training Stage 1: Summarizer	5
Step 8 — TESTING on the held-out 20% (the metrics)	5
8.1 Stage 2A — Presence: the Confusion Matrix (“matrix test”)	5
8.2 Stage 2B — Span extractor	6
8.3 Stage 1 — Summarizer	7
8.4 End-to-end — what actually reaches the user	7
Step 9 — Final scorecard	7
Step 10 — Engineering notes (laptop reality)	8
Files	8

CUAD Legal Document Analysis — Complete Project Report

AI-Powered Legal Document Analysis System · BRAC University, Dept. of CSE Afnan Mazumdar (24141229) · Shoyeb Hasan Sayem (22101386) · Jerin Aktar (22101279) Supervisor: Utsha Kumar Roy · Updated 2026-08-14

A step-by-step account of everything done: data preparation, EDA, training of all three models, the 80/20 train/test split, testing, and the evaluation metrics (including the confusion matrix). Numbers below are the actual results from our runs.

0. Overview

We built a three-model pipeline on the full CUAD dataset (510 contracts, 41 clause categories):

Stage	Model	Job	Test result (20% held-out)
2A	Presence classifier	Which of the 41 clause types are in a contract?	micro-F1 0.776, Accuracy 85.49%
2B	Span extractor	Locate the exact clause text	token-F1 0.763
1	Summarizer	Rewrite a clause in plain English	ROUGE-L 0.775

Numbers below are reproduced by the standalone `test.ipynb`, which loads the saved 80/20-trained models from disk and evaluates them on the held-out 20% test set.

Everything below is the sequence of steps that produced these numbers.

Step 1 — Data source

The official Atticus Project `theatticusproject/cuad` dataset on Hugging Face. Its default `load_dataset()` exposes only raw PDFs (not trainable), so the annotated data was pulled from two repo files:

- `CUAD_v1/master_clauses.csv` — 83-column wide table (used for EDA and label wrangling).
- `CUAD_v1/CUAD_v1.json` — SQuAD format with character offsets (used for full context + span training).

Step 2 — Data preparation (wide → long melt)

`master_clauses.csv` is wide: 41 categories x (clause-text column + answer column). We:

- **Paired columns by position, not by name** — the header `Notice Period To Terminate Renewal- Answer` has a stray space that breaks string matching; position pairing is robust to it.
- **Defined “present” = non-empty clause list** (rather than the Yes/No answer), which works uniformly across the 8 entity categories (dates, parties) and hands us the clause spans for free.

Result: 510 contracts → **20,910 labeled (contract, category) rows**, 32.1% positive overall.

Step 3 — Exploratory Data Analysis (EDA)

- **Severe class imbalance.** Positive rate ranges from `Document Name` 100% and `Parties` 99.8% down to `Source Code Escrow` 2.5%. Median category ~ 23% positive.
- **The rare tail is near few-shot.** Some categories have only a handful of positives in the entire dataset.
- **Contracts are long.** Median ~ 33,000 chars; max > 338,000 chars — about 16x a transformer’s input window.
- **Clauses are short.** Median extracted clause ~ 196 chars. So a clause fits easily in a window; the challenge is *finding* it in a huge document.

Step 4 — The 80/20 train/test split

Following the supervisor’s instruction, the dataset is split **80/20 at the contract level**:

	Contracts	(contract x category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

- **Contract-level** (whole contracts go to one side) so no clause from a training contract leaks into test.
- **Seeded** (SEED=42) and **reused across all three models** — “test” means the same 102 unseen contracts everywhere.
- Split files exported: `data/train_80.csv`, `data/test_20.csv`.

Is the split actually balanced? The aggregate positive rates (32.0% vs 32.2%) agree, but that alone could hide a per-category skew. Comparing all 41 categories directly (`analysis/ch3_chart.py`):

Check	Value
Correlation between train and test positive rates	r = 0.986
Mean absolute difference across the 41 categories	3.4 percentage points

Check	Value
Largest single gap	14.0 pp — <i>Termination For Convenience</i> (47.1% test vs 33.1% train)

The split is well balanced overall. We report the one real outlier rather than smoothing it over: because *Termination For Convenience* is over-represented in test, it carries more weight in the micro-averaged score than its training frequency implies. The split was drawn once at random and never re-drawn.

Step 5 — Training Stage 2A: Presence classifier

5.1 Strong baseline first — TF-IDF + Logistic Regression (trained on 80%)

Classical ML has no length limit, so TF-IDF reads the whole contract. One class-balanced Logistic Regression per category:

Metric	F1
micro (every contract x category equal)	0.775
weighted (by support)	0.777
macro (all 41 categories)	0.572
macro (≥ 10 test positives, 34 cats)	0.666

5.2 The retrieval-ceiling check (why we did not window naively)

Before training a transformer, we measured the **retrieval hit-rate** — can a query find the window that contains the gold clause? Best strategy (category name, top-3) capped at **64%**, i.e. a retrieval-windowed transformer’s recall would be *below* the baseline. Measuring this first saved a doomed training run.

5.3 The fix — sliding-window max-pool (Multiple-Instance Learning)

We trained a **window-level DistilBERT**: (question, window) $\rightarrow$ does this window contain the clause? At inference we run **all** windows and **max-pool** (present if any window fires), which removes the ceiling.

- Windowing: 2,000-char windows, 1,500-char stride $\rightarrow$ a 500-char overlap, which is wider than the median 196-char clause, so no clause can fall between two windows. A median-length contract yields **22 windows**; the corpus mean is **35** and the longest contract gives **226**.
- Training examples: **53,662 windows** (39.2% positive), subsampled to **24,000** for a laptop run.
- Config: DistilBERT-base, 2 epochs, batch 16, LR $2e-5$, max length 256.
- Optimizer (read back from the saved `training_args.bin`): AdamW, **linear decay**, **no warmup**, gradient-norm clipping at **1.0**, 3,000 total steps. Schedule plotted by `analysis/lr_schedule.py`.
- Training loss 0.319 $\rightarrow$ 0.261; validation loss 0.278 $\rightarrow$ 0.257; **~52 min** on Apple MPS.
- Standalone test micro-F1: **0.694** (loses to the baseline — max-pooling inflates false positives: the max is taken over every window, so one spurious window anywhere flips the whole contract).

5.4 The winning model — complementary AND-ensemble

The two models are complementary (transformer wins on semantically subtle categories, baseline on keyword-heavy ones). Combined with parameter-free rules:

Model	micro-F1 (20% test)
Ensemble AND (both must agree)	0.776 (best)
TF-IDF baseline	0.775
Ensemble AVG	0.718
Transformer (max-pool) alone	0.694
Ensemble OR	0.696

The AND-rule filters each model’s uncorrelated false positives → precision up → best F1.

5.5 How much of that ranking is statistically real?

A 0.776-vs-0.775 margin on 102 contracts is one decision in a thousand, so we tested it rather than asserting it. Method: **cluster bootstrap** — resample the 102 *contracts* with replacement (not the 4,182 cells, which are correlated within a contract), recompute micro-F1 for both models on the same resample, 10,000 times. Script: `analysis/bootstrap_ci.py`, `analysis/extra_ci.py`.

Comparison	Difference in micro-F1	95% CI	P(>0)	Verdict
AND-ensemble vs TF-IDF baseline	+0.0014	[-0.0068, +0.0089]	0.626	tie — CI contains zero real
TF-IDF baseline vs transformer	+0.0805	[+0.0626, +0.0992]	1.000	
AND-ensemble vs transformer	+0.0818	[+0.0654, +0.0987]	1.000	real

The two findings are not on equal footing, and this is the honest headline:

- **The ensemble’s win over the baseline is not statistically real.** The interval straddles zero and the ensemble leads in only 62.6% of resamples. Individually: AND 0.776 (95% CI [0.759, 0.792]), TF-IDF 0.775 (95% CI [0.758, 0.791]) — almost entirely overlapping. Read the top two rows of the table above as a **tie**.
- **The transformer’s loss to the baseline is real.** The baseline wins in **all 10,000** resamples.

This does not remove the reason for using the ensemble — that reason was never the 0.001 of micro-F1. The AND rule earns its place through the **error profile** it produces (balanced precision and recall, see §8.1), not through its rank.

5.6 Two controls the bootstrap cannot run

A bootstrap asks “could this gap be sampling noise?” It cannot ask “was the transformer handicapped?” Both alternative explanations were tested (`analysis/presence_run.py`, `analysis/compare_runs.py`).

Control A — training budget. The baseline is fitted on all 408 contracts in full; the transformer saw 24,000 of 53,662 windows. Retrained on **all 53,662** (140.7 min), everything else identical:

Transformer training windows	micro-F1	Precision	Recall
24,000 (44.7%)	0.6943	0.5620	0.9080
53,662 (100%)	0.6939	0.5546	0.9266

Difference **-0.0004**, CI [-0.0098, +0.0095]; gap to TF-IDF holds at **+0.0809**. More data made the model fire *more*, not better — recall up, precision down — which is the max-pooling failure mode over ~35 windows. **The deficit is architectural, not a budget artifact.**

Control B — random seed. Three runs at the 24,000 budget:

Model	Seed 42	Seed 43	Seed 44	Mean	SD	Range
Transformer	0.6943	0.6928	0.7082	0.6984	0.0085	0.0154
AND ensemble	0.7761	0.7774	0.7803	0.7780	0.0022	0.0042

The ensemble’s SD (0.0022) and range (0.0042) both **exceed its +0.0014 margin** — reseeding moves the score further than the result being claimed for it. The baseline’s lead over the transformer is 9.5 SDs. The ensemble is also 4x less seed-variable than the transformer alone, which is a better argument for it than its rank.

5.7 Does the AND rule punish rare categories?

Splitting the 41 categories by training frequency (`analysis/rare_tail.py`):

Group	Model	Precision	Recall	F1	TP	FN
Rarest 10 (70 test pos.)	Transformer	0.261	0.500	0.343	35	35
	AND ensemble	0.586	0.243	0.343	17	53
Other 31 (1,278 test pos.)	Transformer	0.582	0.930	0.716	1,189	89
	AND ensemble	0.776	0.810	0.792	1,035	243

On common categories the conjunction gains +0.077 F1. **On the rare tail it nets exactly zero** — 32.5 points of precision bought with 25.7 points of recall, detections cut from 35/70 to 17/70. All of the ensemble’s aggregate benefit comes from common categories. *Most Favored Nation* sits in this tail, is a **High-risk** category, and scores 0.000 — which matters more for a signer than the aggregate does.

Step 6 — Training Stage 2B: Span extractor

- SQuAD-style DistilBertForQuestionAnswering. Char offsets → token start/end via the tokenizer’s offset mapping; a 1,200-char focused window is re-centered on each answer so it fits in 320 tokens.
- **11,156 train / 2,667 test** examples; trained on **8,000**; 2 epochs, batch 16, LR 3e-5; ~**23 min** MPS.

Step 7 — Training Stage 1: Summarizer

- Clause → one plain-English sentence. CUAD has no plain-language targets, so we built a **41-template seed** (one hand-written gold summary per category, weaker-party framing).
- FLAN-T5-small, **11,156 pairs** (trained on 4,000); 2 epochs, batch 8, LR 3e-4.
- Training loss 0.141 → 0.085; validation loss 0.096 → 0.075; ~**8 min on CPU** (`use_cpu=True` to avoid MPS OOM).

Step 8 — TESTING on the held-out 20% (the metrics)

All numbers below are on the **102 unseen test contracts**.

8.1 Stage 2A — Presence: the Confusion Matrix (“matrix test”)

AND-Ensemble evaluated on all **4,182 (contract x category) test predictions**:

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,523	FP = 311
Actually Present	FN = 296	TP = 1,052

Metrics derived from the confusion matrix:

Metric	Formula	Value
Accuracy	$(TP+TN) / \text{all}$	85.49%
Precision	$TP / (TP+FP)$	77.18%
Recall (Sensitivity)	$TP / (TP+FN)$	78.04%
Specificity	$TN / (TN+FP)$	89.03%
F1 Score	$2 \cdot P \cdot R / (P+R)$	0.7761

Per-category F1 (top of the table):

Category	Test positives	Precision	Recall	F1
Document Name	102	1.000	1.000	1.000
Parties	101	0.990	1.000	0.995
Governing Law	90	0.956	0.967	0.961
Agreement Date	93	0.929	0.989	0.958
Expiration Date	84	0.914	0.881	0.897
License Grant	56	0.907	0.875	0.891
Anti-Assignment	74	0.831	0.932	0.879
Effective Date	73	0.831	0.877	0.853
Cap On Liability	62	0.879	0.823	0.850

Categories with ≤ 7 test positives (e.g. Source Code Escrow, Most Favored Nation) score near 0 — too few examples to learn from in a 510-contract dataset, not a model failure.

Why the ensemble is worth keeping despite §5.5. Precision (77.18%) and recall (78.04%) come out balanced, which is exactly what the AND rule was for: raw max-pooling trades precision away, and requiring agreement wins it back without costing recall. Specificity of 89.03% means the system rarely claims a clause that is not there — in a legal-review assistant that is the more damaging error.

8.2 Stage 2B — Span extractor

Evaluated on **2,667 answer-bearing test windows**:

Metric	Score	95% CI (by contract)	95% CI (by clause)
token-F1	0.763	[0.740, 0.782]	[0.751, 0.774]
Overlap (F1 ≥ 0.5)	82.7%	[79.7%, 85.1%]	[81.3%, 84.1%]
Exact Match	35.5%	[32.5%, 38.6%]	[33.7%, 37.3%]

When the clause is in front of it, the model captures most of the right text ~8 times in 10. The same cluster bootstrap used for presence is applied here (`analysis/span_ci.py`); resampling contracts gives an interval ~2x wider than treating clauses as independent. **These numbers assume an answer-bearing window** — Step 8.4 measures what happens when Stage 2A picks the window.

8.3 Stage 1 — Summarizer

The 41 targets were **written by us**, not annotated. Scoring the same **150 test clauses** against a second, clause-specific reference set separates template reproduction from summarization (`analysis/summ_pilot.py`):

System	Reference set	ROUGE-L	95% CI
Fine-tuned	41 category templates	0.775	[0.718, 0.833]
Fine-tuned	clause-specific (pilot)	0.116	[0.105, 0.128]
Zero-shot	clause-specific (pilot)	0.299	[0.275, 0.322]
<i>Template alone, no model</i>	clause-specific (pilot)	0.115	[0.104, 0.127]

The fine-tuned model emits one of the 41 templates **verbatim on 100% of clauses** (the correct one 70% of the time). The bare template with no model scores the same 0.115 — the model adds nothing measurable over a lookup. This is a **classification-to-template** system and is described as such throughout; it is not free-text summarization. Pilot references are LLM-drafted (independent of the model, not gold annotation).

8.4 End-to-end — what actually reaches the user

Stage-wise scores do not compose. Running all three stages in series over the 1,348 gold clauses in the 102 test contracts (`analysis/e2e.py`):

Stage	Condition	Surviving	Share
—	gold clauses	1,348	100.0%
2A Presence	category detected	1,052	78.0%
2B Span	overlap with gold at F1 ≥ 0.5	293	21.7%
Risk	correct level	293	21.7%
End-to-end		293	21.7% (CI [19.6%, 23.9%])

Component multiplication predicts ~64%; the pipeline delivers a third of it. Of the 1,052 detected clauses, the selected window contains the gold text only **68.3%** of the time (max-pooling was trained to detect, not to locate), and even then extraction succeeds **39.7%** vs 82.7% isolated — because training re-centred every focus window on the answer, narrowing the distribution the model can read. Mean token-F1 falls 0.763 $\rightarrow$ 0.386. Both causes are fixable without a larger model.

For the app: the risk badge and category are far more reliable than the quoted clause text.

Step 9 — Final scorecard

Stage	Model	Train	Test	Metric	Score
2A Presence	AND-ensemble (TF-IDF + DistilBERT)	408 contracts	102 contracts	micro-F1	0.776
2A Presence	AND-ensemble	408	102	Accuracy	85.49%
2A Presence	AND-ensemble	408	102	Precision / Recall	77.18% / 78.04%
2B Span	DistilBERT- QA	408	102	token-F1 (given the window)	0.763

Stage	Model	Train	Test	Metric	Score
2B Span	DistilBERT-QA	408	102	token-F1 (window from 2A)	0.386
2B Span	DistilBERT-QA	408	102	Exact Match	35.5%
1 Summarizer	FLAN-T5-small	408	102	ROUGE-L (41 templates)	0.775
1 Summarizer	FLAN-T5-small	408	102	ROUGE-L (clause-specific)	0.116
Full pipeline	all three stages	408	102	gold clauses surviving	21.7%

Read the presence row with §5.5 in hand. The AND-ensemble’s 0.776 is not statistically separable from the TF-IDF baseline’s 0.775 on this test set. What *is* separable — and is the result we stand behind — is that both beat the windowed transformer alone (0.694) by a wide, significant margin.

And read the whole table with §8.4 in hand. These are component scores measured with the other stages held out of the way. End to end only **21.7%** of gold clauses survive all three stages. That is the number describing what a user experiences.

Step 10 — Engineering notes (laptop reality)

- **Apple MPS, no CUDA** (torch 2.8, transformers 5.x). DistilBERT + FLAN-T5-small throughout (DeBERTa-v3 is numerically unstable on MPS). On a CUDA GPU, swap in DeBERTa-v3-base / flan-t5-base.
- **MPS memory does not release in-process** → summarizer trained on CPU with `use_cpu=True`.
- **transformers 5.x API**: `processing_class=...` (not `tokenizer=...`); `fp16/bf16=False`; `dataloader_pin_memory=False`
- **Leakage discipline** held throughout: contract-level split; retrieval queries use only the category/question; gold offsets label training windows but never build the input or pick the inference window; the ensemble is parameter-free.
- **One thing we did not keep.** The intermediate checkpoint directories were cleared after training, so per-step **gradient-norm traces were not retained** — only the final weights and the loss values above. The learning-rate schedules can be reconstructed exactly (from the saved `training_args.bin`) and are plotted, but the gradient-norm diagnostic would need a re-run (~90 min). Noted as an omission rather than glossed over.

Files

```

clausify-research/
|--- README.md           - start here: what this is and where to look
|--- thesis/             - the B.Sc. thesis (LaTeX source + compiled PDF)
|   |--- main.tex / main.pdf - 55 pages, 8 chapters + appendices
|   |--- chapters/        - one .tex per chapter
|   `--- figures/         - generated plots (TikZ diagrams live in the chapters)
|--- PROJECT_REPORT.md / .pdf - this complete step-by-step report
|--- REPORT.md / .pdf     - findings-focused report (decisions & caveats)
|--- METHODOLOGY.md / .pdf - design-decision record
|--- analysis/           - scripts that generate the reported statistics
|   |--- bootstrap_ci.py  - cluster bootstrap CI (§5.5) + point estimates
|   |--- extra_ci.py      - the three paired model contrasts

```

```

|   |--- ch3_chart.py           - train-vs-test split distribution figure
|   |--- lr_schedule.py        - learning-rate schedule figure
|   `--- *.json                - saved numeric results
|--- data/
|   |--- train_80.csv           - 80% training split (16,728 rows, 408 contracts)
|   `--- test_20.csv           - 20% test split (4,182 rows, 102 contracts)
|--- notebooks/
|   |--- README.md             - cell-by-cell notebook guide
|   |--- train.ipynb           - training half
|   |--- test.ipynb            - testing half (regenerates every number above)
|   |--- eda.ipynb             - the original combined notebook
|   |--- artifacts/            - the split + the fitted TF-IDF baseline
|   `--- outputs/              - the three trained model checkpoints

```

Reproducing the numbers. `notebooks/test.ipynb` loads the saved models from disk and regenerates every metric in this report. The bootstrap intervals in §5.5 come from `analysis/bootstrap_ci.py`, which independently reproduces the same point estimates (0.776 / 0.775 / 0.718 / 0.696 / 0.694) and the same confusion matrix (TN=2523, FP=311, FN=296, TP=1052) before resampling.